UCAC-ICAM

Institut Catholique des Arts et Metiers

AgriSmart

Systeme de Gestion Agricole Intelligent

Projet Integrateur X2 · 2025

Document	Documentation Technique Complete
Version	1.0
Date	2025
Equipe	Groupe de 6 etudiants
Statut	En cours de realisation

1. Besoins du Projet

1.1 Contexte

L'agriculture locale fait face a des defis croissants : optimisation des ressources en eau, surveillance des conditions climatiques, automatisation des taches repetitives et securisation des equipements. Dans ce contexte, la cooperative agricole locale (CAL) mandate UCAC-ICAM pour developper un prototype de systeme de gestion agricole intelligent base sur des technologies IoT.

Ce systeme, baptise AgriSmart, connecte les exploitations agricoles a une plateforme intelligente permettant une surveillance en temps reel et une automatisation des actions critiques.

1.2 Objectifs

- Collecter des donnees environnementales en temps reel via des capteurs IoT
- Automatiser des actions agricoles critiques (irrigation, ventilation, eclairage)
- Offrir une interface web intuitive pour la visualisation et le controle
- Securiser l'architecture et les donnees du systeme
- Fournir un historique complet des actions et mesures exportable en CSV

1.3 Parties Prenantes

Partie prenante	Role	Besoins principaux
Agriculteurs	Utilisateurs finaux	Suivi cultures, alertes, controle manuel
Administrateurs	Gestion de la plateforme	Comptes, seuils, configuration
Ingenieurs IoT	Infrastructure	Connectivite ESP32, MQTT/HTTP
Enseignants UCAC-ICAM	Evaluation pedagogique	Conformite aux modules enseignes

1.4 Composantes du Systeme

1.4.1 Systeme IoT

Cinq capteurs collectent les parametres environnementaux :

- Humidite du sol — YL-69 (%)
- Temperature ambiante — DHT22 (degres Celsius)
- Taux de CO_2 — SEN0159 (ppm)
- Luminosite — BH1750 (lux)
- Niveau reservoir — capteur de niveau (%)

Trois actionneurs executent les actions :

- Pompe d'irrigation (relais 5V)
- Ventilateur de serre
- Systeme d'eclairage

1.4.2 Application Web (AE)

- Authentification avec roles admin / viewer
- Dashboard temps reel avec graphiques et sparklines
- Controle manuel des actionneurs (admin uniquement)
- Configuration des seuils d'alerte
- Gestion des utilisateurs (creation par admin uniquement)
- Historique avec export CSV
- Alertes temps reel via WebSocket
- Interface multilingue francais / anglais

1.4.3 Application d'Automatisation (AA)

- Interface CLI et scripts batch pour actions en masse
- Journalisation de chaque execution en base de donnees
- Independant de l'interface web et du microcontroleur

1.5 Fonctionnalites Generales

Fonctionnalite	Description	Acces
Surveillance temps reel	Affichage live des valeurs capteurs	Admin + Viewer
Alertes automatiques	Notification si seuil depasse	Admin + Viewer
Controle actionneurs	On/Off pompe, ventilation, eclairage	Admin uniquement
Historique & export	Consultation et export CSV	Admin + Viewer
Gestion utilisateurs	Creation et desactivation comptes	Admin uniquement
Configuration seuils	Valeurs min/max par capteur	Admin uniquement
Logs scripts CLI	Traçabilite des executions batch	Admin uniquement
Multilingue	Francais et anglais	Admin + Viewer

1.6 Contraintes de Securite

- Mots de passe haches avec bcrypt
- Access tokens en memoire uniquement (jamais localStorage)
- Refresh tokens via cookie httpOnly
- ESP32 authentifie par cle API statique dans l'en-tete HTTP
- Communications ESP32 via HTTPS / MQTTS
- Chaque execution CLI journalisee en base de donnees

- Roles admin et viewer avec droits strictement definis
- Schema public PostgreSQL verrouille — acces PUBLIC revoque

2. Structure de la Base de Donnees

La base de donnees AgriSmart est implementee sous PostgreSQL. Elle comprend 14 tables organisees en 5 domaines fonctionnels, construites par 8 fichiers SQL executes via setup.sh.

> Base : agriculture_intelligente | SGBD : PostgreSQL | Utilisateur app : agri_app | Migration : agri_migration

2.1 Domaine Authentification & Sessions

Ce domaine couvre la gestion des comptes, sessions et tokens. Les comptes sont crees exclusivement par un administrateur — aucune inscription publique.

Table USERS

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique (gen_random_uuid)
username	VARCHAR(50)	UNIQUE NOT NULL	Nom de connexion
email	VARCHAR(150)	UNIQUE NOT NULL	Adresse email
password_hash	TEXT	NOT NULL	Mot de passe hache bcrypt
role	ENUM	NOT NULL	admin ou viewer
language	ENUM	DEFAULT fr	fr ou en
is_active	BOOLEAN	DEFAULT TRUE	Compte actif / desactive
created_by	UUID	FK -> USERS	Admin ayant cree le compte (self-ref)
created_at	TIMESTAMPTZ	DEFAULT NOW()	Date de creation
updated_at	TIMESTAMPTZ	AUTO (trigger)	Derniere modification

Tables de Tokens

Deux tables distinctes pour une securite optimale :

Caracteristique	ACCESS_TOKENS	REFRESH_TOKENS
Duree	15 minutes	7 a 30 jours
Usage	Chaque requete HTTP	Obtenir un nouvel access token
Stockage frontend	Memoire JavaScript	Cookie httpOnly
Rotation	Non (expire)	Oui — revoke + nouveau a chaque usage

Caracteristique	ACCESS_TOKENS	REFRESH_TOKENS
Champ cle	revoked_at	last_used_at + revoked_at

Table PASSWORD_RESET_TOKENS

Tokens a usage unique generes par admin. Marques utilises apres consommation, jamais supprimes (audit). Champs : id, user_id (FK), token_hash, expires_at, used_at, created_at.

2.2 Domaine Capteurs

Table SENSORS

Decrit chaque capteur physique. La desactivation ne supprime pas l'historique.

Type ENUM	Modele	Unite	Location type
humidity	YL-69	%	Parcelle
temperature	DHT22	C	Serre
co2	SEN0159	ppm	Serre
luminosity	BH1750	lux	Parcelle
water_level	Personnalise	%	Reservoir

Table SENSOR_READINGS

Table la plus volumineuse. Chaque mesure ESP32 est horodatee. L'index composite (sensor_id, measured_at DESC) est critique pour les performances du dashboard.

> Index critique : CREATE INDEX idx_sensor_readings_sensor_time ON sensor_readings(sensor_id, measured_at DESC)

Table SENSOR_STATUS_LOGS

Historise les changements d'etat (online / offline / error). Alimente automatiquement par trigger lors de la modification de is_active sur SENSORS.

2.3 Domaine Seuils & Actionneurs

Table THRESHOLDS

Type capteur	Min defaut	Max defaut	Alerte generee
humidity	30 %	80 %	low_humidity si valeur < 30
temperature	15 C	35 C	high/low_temperature

Type capteur	Min defaut	Max defaut	Alerte generee
co2	NULL	1000 ppm	high_co2 si valeur > 1000
luminosity	200 lux	NULL	low_luminosity si valeur < 200
water_level	20 %	NULL	low_water_level si valeur < 20

Table ACTUATORS

Decrit chaque actionneur physique. Le champ status (on/off) et last_triggered_at sont mis a jour automatiquement par trigger apres chaque insertion dans ACTIONS.

2.4 Domaine Actions & Alertes

Table ACTIONS

Journalise chaque evenement de controle. Le champ source distingue les origines :

Source	Declencheur	triggered_by
web	Admin via interface	UUID utilisateur
cli	Script batch / CLI	UUID ou NULL
auto	Systeme (seuil franchi)	NULL

Table ALERTS

Generees automatiquement par le trigger fn_check_threshold_and_alert() apres chaque insertion dans SENSOR_READINGS. Trois niveaux de severite :

Severite	Types d'alertes	Actionneur associe
high	low_humidity, high/low_temperature, low_water_level	Pompe ou ventilation
medium	high_co2, low_luminosity	Ventilation ou eclairage
low	sensor_failure	Aucun

2.5 Domaine Traçabilite

Table SCRIPT_LOGS

Journalise chaque execution du script batch CLI. Stocke la commande exacte, le resultat, la source et l'horodatage. Exigence de securite du projet.

Table EXPORTS

Trace chaque generation CSV depuis l'application. Stocke le type, les filtres (JSONB) et le chemin du fichier genere.

2.6 Triggers Automatiques

Trigger	Table	Evenement	Action automatique
trg_users_updated_at	USERS	BEFORE UPDATE	Met a jour updated_at
trg_session_activity	REFRESH_TOKENS	AFTER UPDATE	Met a jour last_activity_at (SESSIONS)
trg_actuator_on_action	ACTIONS	AFTER INSERT	Met a jour status + last_triggered_at (ACTUATORS)
trg_check_threshold	SENSOR_READINGS	AFTER INSERT	Genere une alerte si seuil depasse
trg_log_sensor_status	SENSORS	AFTER UPDATE	Journalise les changements is_active
trg_thresholds_updated_at	THRESHOLDS	BEFORE UPDATE	Met a jour updated_at

2.7 Vues Metier

Vue	Usage	Tables jointes
v_latest_readings	Dashboard — cartes capteurs (valeur actuelle)	SENSOR_READINGS + SENSORS
v_sensor_readings	Dashboard — historique par capteur	SENSOR_READINGS + SENSORS
v_active_alerts	Dashboard — zone alertes non traitees	ALERTS + SENSORS + ACTUATORS
v_action_history	Page historique — onglet actions	ACTIONS + USERS + ACTUATORS
v_alert_history	Page historique — onglet alertes	ALERTS + SENSORS + ACTUATORS + USERS

2.8 Fichiers SQL

Fichier	Contenu	Execute par
00_create_database.sql	Creation BDD + roles agri_app / agri_migration	Superuser
00_reset.sql	Suppression de tous les objets (ordre	agri_migration

Fichier	Contenu	Execute par
	inverse)	
01_extensions.sql	pgcrypto pour gen_random_uuid()	agri_migration
02_enums.sql	Tous les types ENUM personnalises	agri_migration
03_tables.sql	14 tables avec contraintes et FK	agri_migration
04_indexes.sql	Index de performance	agri_migration
05_views.sql	5 vues metier	agri_migration
06_triggers.sql	6 triggers d'automatisation	agri_migration
07_sample_data.sql	Donnees de test (dev uniquement)	agri_migration
08_permissions.sql	Droits minimum pour agri_app	Superuser
setup.sh	Script d'execution ordonnee	Shell Bash

3. Definition du Backend

3.1 Stack Technique

Composant	Technologie	Role
Framework	Django 4.x + Django REST Framework	API REST, logique metier, ORM
Temps reel	Django Channels (ASGI)	WebSocket pour donnees live
Base de donnees	PostgreSQL via psycopg2	Persistence des donnees
Authentification	djangorestframework-simplejwt	Gestion JWT access/refresh
Serveur ASGI	Daphne	Gere HTTP + WebSocket simultanement
Cache / Channels	Redis	Layer de messagerie pour Channels
CORS	django-cors-headers	Autorisation du frontend React
Config secrets	python-decouple	Variables d'environnement (.env)
Hebergement	Railway ou Render	Deploiement cloud

3.2 Architecture Generale

Le backend fonctionne sur deux couches de communication simultanees :

> REST API : Authentification, CRUD, historique, exports, gestion utilisateurs. WebSocket (Django Channels) : Diffusion temps reel des mesures capteurs et alertes vers le frontend des reception de l'ESP32.

Flux de donnees principal :

- L'ESP32 envoie les mesures via POST /api/iot/readings/ avec une cle API statique
- Django enregistre la mesure dans SENSOR_READINGS
- Le trigger PostgreSQL verifie les seuils et cree une alerte si necessaire
- Django Channels diffuse sensor.reading (et alert.new si applicable) via WebSocket
- Le frontend React met a jour le dashboard en temps reel sans rechargement

3.3 Structure du Projet Django

Dossier / Fichier	Contenu
config/settings/base.py	Parametres communs (BDD, apps, middleware, JWT)
config/settings/development.py	Overrides locaux (DEBUG=True, CORS permissif)

Dossier / Fichier	Contenu
config/settings/production.py	Overrides cloud (HTTPS, CORS strict, HSTS)
config/asgi.py	Point d'entree ASGI — route HTTP vs WebSocket
config/urls.py	Configuration racine des URLs
apps/auth/	Login, logout, refresh token, reset mot de passe
apps/users/	CRUD utilisateurs (admin uniquement)
apps/sensors/	Capteurs, mesures, seuils, logs statut
apps/actuators/	Actionneurs et actions
apps/alerts/	Alertes et acquittement
apps/script_logs/	Logs d'execution CLI
apps/exports/	Generation et telechargement CSV
apps/iot/	Endpoint ingestion donnees ESP32
websocket/consumers.py	Consumers WebSocket (Channels)
websocket/routing.py	Routage WebSocket
websocket/events.py	Types d'evenements envoyes au frontend

3.4 Endpoints REST API

Authentification — /api/auth/

Methode	Endpoint	Description	Acces
POST	/login/	Connexion — retourne access + refresh tokens	Public
POST	/logout/	Revoque session et tokens	Authentifie
POST	/token/refresh/	Nouveau access token via refresh token	Authentifie

Utilisateurs — /api/users/

Methode	Endpoint	Description	Acces
GET	/	Liste tous les utilisateurs	Admin
POST	/	Cree un utilisateur	Admin
GET	/:id/	Detail d'un utilisateur	Admin
PATCH	/:id/	Modifie role, langue, statut actif	Admin
POST	/:id/reset-password/	Genere token de reinitialisation	Admin

Capteurs — /api/sensors/

Methode	Endpoint	Description	Acces
GET	/	Liste tous les capteurs	Authentifie
GET	/:id/readings/	Historique mesures (filtre date)	Authentifie
GET	/:id/latest/	Derniere mesure	Authentifie
PATCH	/:id/	Activer / desactiver capteur	Admin

Seuils, Actionneurs, Alertes

Methode	Endpoint	Description	Acces
GET	/api/thresholds/	Liste tous les seuils	Authentifie
PATCH	/api/thresholds/:type/	Modifie min/max d'un capteur	Admin
GET	/api/actuators/	Liste actionneurs avec statut	Authentifie
POST	/api/actuators/:id/action/	Declenche action on/off	Admin
GET	/api/alerts/	Liste alertes (filtres severity/date)	Authentifie
POST	/api/alerts/:id/acknowledge/	Acquitte une alerte	Admin

Logs, Exports & IoT

Methode	Endpoint	Description	Acces
GET	/api/script-logs/	Liste logs CLI	Admin
POST	/api/script-logs/	Enregistre execution script	Cle API
POST	/api/exports/	Genere et telecharge CSV	Authentifie
GET	/api/exports/	Liste les exports passes	Authentifie
POST	/api/iot/readings/	ESP32 envoie une mesure	Cle API statique

3.5 Evenements WebSocket

Le frontend se connecte a ws://domaine/ws/dashboard/ apres authentification. Evenements recus :

Evenement	Payload cle	Declencheur
sensor.reading	sensor_id, value, measured_at, unit	Chaque mesure ESP32
alert.new	Objet Alert complet	Seuil franchi — trigger PostgreSQL
alert.acknowledged	alert_id, acknowledged_by, acknowledged_at	Admin acquitte alerte
actuator.updated	actuator_id, status, last_triggered_at	Action declenchee

Evenement	Payload cle	Declencheur
sensor.status	sensor_id, status, reason	Capteur online / offline

3.6 Securite du Backend

Mesure de securite	Implementation technique
Hachage mots de passe	Django bcrypt hasher
Access token	JWT HS256, 15 min, stockage memoire frontend
Refresh token	JWT longue duree, cookie httpOnly, rotation a chaque usage
Auth ESP32	Cle API statique dans header X-API-Key
CORS	Liste blanche stricte en production (domaine frontend uniquement)
HTTPS	Obligatoire en production via Daphne + reverse proxy
Permissions Django	IsAuthenticated + IsAdminUser par viewset
Rate limiting	django-ratelimit sur /api/auth/login/ (5 tentatives/min)

3.7 Deploiement Cloud

Service	Technologie	Notes
Web service	Daphne (ASGI)	HTTP REST + WebSocket simultanement
Base de donnees	PostgreSQL addon	Provisionnee par Railway / Render
Cache	Redis addon	Requis pour layer Django Channels
Variables env	.env / dashboard cloud	DATABASE_URL, SECRET_KEY, REDIS_URL, API_KEY_ESP32

> Procfile (commande de demarrage) : web: daphne -b 0.0.0.0 -p $PORT config.asgi:application

3.8 Ordre de Developpement

Etape	Composante	Dependances
1	config/ — settings, ASGI, URLs	Aucune
2	Modeles Django (models.py)	Base de donnees prete
3	apps/auth/ — login, logout, refresh	USERS, SESSIONS, TOKENS
4	apps/users/ — gestion comptes	apps/auth/

Etape	Composante	Dependances
5	apps/sensors/ — capteurs, mesures	Modeles SENSORS, READINGS, THRESHOLDS
6	apps/actuators/ — actionneurs	Modeles ACTUATORS, ACTIONS
7	apps/alerts/ — alertes	apps/sensors/ + apps/actuators/
8	apps/script_logs/ — logs CLI	Modele SCRIPT_LOGS
9	apps/exports/ — CSV	Toutes les apps precedentes
10	websocket/ — Channels	Redis + toutes les apps
11	apps/iot/ — ingestion ESP32	apps/sensors/ + websocket/
12	Fichiers de deploiement	Projet complet